Your users can also supply their own command-line arguments to your Rake tasks by setting the SPEC_OPTS environment variable. For example, they can get documentation-style output from your spec:unit task by calling it like so:

```
$ rake spec:unit SPEC_OPTS='-fd'
```

If you're developing a web app, you probably keep test-only gems like RSpec off your production servers. To use Rake in such an environment, you'll need to handle the case when RSpec isn't available:

A1-rspec-and-wider-ecosystem/02/expense_tracker/Rakefile

```
begin
  require 'rspec/core/rake_task'
  RSpec::Core::RakeTask.new(:spec)
rescue LoadError
  puts 'Spec tasks not defined since RSpec is unavailable'
end
```

Now, you'll be able to use Rake to run specs on your development machine, and to run deployment tasks (such as compiling assets) in your production environment.

Using Parts of RSpec With Other Test Frameworks

Sometimes, you'll want to use the powerful test doubles available in rspec-mocks, or the composable matchers provided by rspec-expectations, in a project where the rest of RSpec isn't a good fit. For instance, you may already have an extensive test suite written in Minitest, or you may be writing acceptance tests in Cucumber.[2,3]

Both of these parts of RSpec are easy to use with other test frameworks, or even without a test framework at all. In fact, you've already done so in *Parts of an Expectation* and throughout *Understanding Test Doubles*.

If you're using Minitest, RSpec provides a couple of conveniences for you, including the following:

- Reporting unmet expectations correctly as Minitest assertion failures
- Making sure RSpec's expect method doesn't clash with Minitest's method of the same name
- Verifying message expectations you've set on mock objects

2. https://github.com/seattlerb/minitest
3. https://cucumber.io